Kuwait University

Department of Computer Science
College of Science

CyberCrypto

Cryptography Project

Author: Abeer Alsafran
224125356

May 4, 2026

1 Introduction

In this project, we will design and implement an advanced secure communication system that combines theoretical cryptographic foundations with practical system-level security design. The project will focus on the following key areas:

1. Security reasoning and a formal understanding of cryptographic principles. 2. Cryptographic design choices and their trade-offs. 3. Awareness of potential attacks and resistance strategies. 4. Performance analysis and scalability considerations. 5. Connections to current research and real-world applications. 6. Simulation of a secure end-to-end communication protocol for two or more parties under realistic threat models.

2 Cryptographic Background

2.1 Symmetric encryption

Cryptography focuses on securing communication against malicious adversaries. A secret key in cryptography is a piece of information—typically a string of bits—that an encryption algorithm uses to transform plaintext into ciphertext and vice versa. In symmetric encryption, the same secret key is used for both the encryption and decryption of data, which is why it's often referred to as a shared key or private key.

The security of symmetric encryption relies entirely on keeping this key confidential. If an attacker gains access to the key, they can decrypt all messages encrypted with it.

Due to its speed and efficiency, symmetric encryption is widely used to secure communications. Some common use cases include:

1. **File and Disk Encryption**: Symmetric encryption is the preferred method for securing files, databases, and entire drives because of its robust performance and simplicity.

2. **Bulk Data Encryption**: For encrypting large volumes of data, symmetric encryption is the method of choice due to its faster processing time compared to asymmetric encryption.

3. **Hybrid Algorithms**: While asymmetric encryption is essential for securing keys and verifying identities, it is not the best choice for encrypting data. Hybrid systems combine the strengths of both: they use asymmetric encryption for key exchange and symmetric encryption for the actual data encryption.

Here are some notable symmetric encryption standards:

- **AES (Advanced Encryption Standard)**: A widely adopted symmetric encryption standard endorsed by NIST for national and industrial use. It is available in 128-bit, 192-bit, and 256-bit key sizes and offers high performance and security.

- **DES (Data Encryption Standard)**: Formerly popular, DES is now considered obsolete due to its vulnerability to brute-force attacks. It uses a 56-bit key size and has been replaced by more secure alternatives like AES and 3DES.

- **Triple DES (3DES)**: An improved version of DES, which applies the DES algorithm three times to each data block. Although stronger than DES, it is slower and less efficient than AES.

2.2 Asymmetric encryption

Asymmetric encryption, also known as public-key cryptography, secures data using a pair of keys: a public key for encryption and a private key for decryption. The public key can be shared openly, while the private key must remain confidential. This eliminates the need for a secure key exchange, which is a common vulnerability in symmetric encryption.

Common Algorithms:

- **RSA**: Based on the difficulty of factoring large prime numbers; widely used in HTTPS, VPNs, and secure email.
- **ECC**: Utilizes elliptic curves; provides strong security with shorter keys, making it ideal for IoT and mobile applications.
- **DSA**: Specializes in creating digital signatures.

Advantages: 1- Enhanced security: There is no need to share private keys. 2- Authentication and non-repudiation: Digital signatures verify the identity of the sender. 3- Secure key exchange: Often employed to exchange symmetric keys for faster bulk encryption.

Limitations: 1- Slower than symmetric encryption. 2- Higher computational cost. 3- Vulnerable to quantum computing threats, prompting research into post-quantum cryptography.

Due to the complex nature of the encryption process, asymmetric encryption requires more computational resources, such as CPU power and memory. This can be problematic for devices with limited capabilities. Additionally, asymmetric encryption is significantly slower than symmetric encryption because it involves intricate mathematical operations. Encrypting large volumes of data can be inefficient, making this method less suitable for bulk data encryption. Furthermore, algorithms like RSA and ECC are potentially vulnerable to attacks from quantum computers, as these machines could efficiently solve problems such as factoring large numbers, thereby compromising current asymmetric encryption systems.

2.3 Diffie-Hellman

The Diffie-Hellman Key Exchange is a cryptographic protocol that enables two parties to securely establish a shared secret key over an insecure communication channel. Proposed by Whitfield Diffie and Martin Hellman in 1976, it serves as a foundational element for many cryptographic protocols. The security of the Diffie-Hellman key exchange relies on the difficulty of solving the discrete logarithm problem. Even if an eavesdropper intercepts the public keys, they cannot easily determine the shared secret without access to the private keys.Perfect Forward Secrecy (PFS) ensures that even if the private keys are compromised, past communications remain secure. This is accomplished by generating unique session keys for each session, which are not reused.

2.4 Hashing

Hashing is the process of converting a key or a string of characters into a different value. This value is typically shorter and has a fixed length, making it easier to locate or use the original string. One of the most common applications of hashing is in the creation of hash tables. A hash table stores key-value pairs in a list that can be accessed via its index. Since the number of keys and value pairs can be unlimited, the hash function maps these keys to the size of the table. The resulting hash value then serves as the index for a specific element.

A hash function produces new values based on a mathematical algorithm, which is referred to as a hash value or simply a hash. To ensure that the original key cannot be reverse-engineered from the hash, a reliable hash function employs a one-way hashing algorithm. Various encryption algorithms are used, including MD5, SHA-256, and SHA-512. Each of these algorithms has distinct characteristics and levels of security, and the specific requirements of the application will determine which algorithm is appropriate.

2.5 Authentication

Authentication is the process that organizations use to ensure that only authorized individuals, services, and applications can access their resources. This process is crucial for cybersecurity, as unauthorized access is a primary goal of malicious actors, who often attempt to gain entry by stealing usernames and passwords from legitimate users.

The authentication process consists of three main steps:

1. **Identification**: Users identify themselves by providing a username.

2. **Authentication**: Users must prove their identity by entering a password—something that only they should know. To enhance security, many organizations also require additional verification methods, such as providing something they possess (like a phone or token device) or something intrinsic to them (such as a fingerprint or facial scan).

3. **Authorization**: The system checks to ensure that users have the appropriate permissions for the resources they are trying to access.

3 System Design

3.1 Architecture diagram

The flow diagram in Figure 1 shows the flow of the CyberCrypto module for handshake, message encryption, and authentication.

3.2 Protocol description

Figure 2 illustrates the handshake process under the following assumptions:

- Client-Server Connection over Socket.

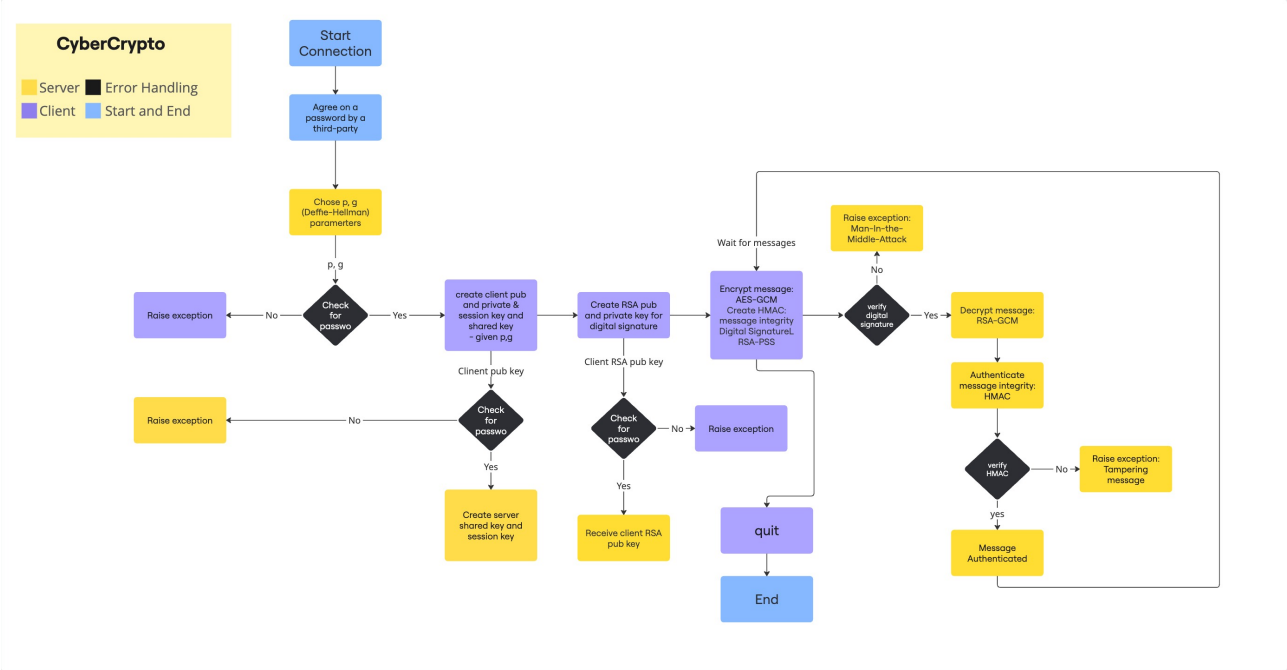


Figure 1: CyberCrypto Flow Digram

- Agree on a password through a third-party channel.

4 Implementation Details

4.1 Hanshake and Secure Channel

The first step was to create a network socket for the connection between the client and the server. Through a socket we have create a secure channel for exchanging the session key. Assume that the client and the server have agreed on a secret name that can be appended to the beginning of the message, such as "SALAM", in the first communication to securely exchange the public keys and the Diffie-Hellman parameters (p, g). And this agreement has been made through a third party, or perhaps in person.

The handshake is done using the Diffie-Hellman key exchange mechanism, where the server selects two parameters (p,g) and generates the server's public key, and then passes these arguments to the client. Based on the parameters, the server sends the client its public key, and shares the public key with the server.

4.2 Algorithms used

- **Diffie-Hellman**
- **RSA-SHA-256**
- **AES-GCM**
- **HMAC**

4.3 Key Generation

We employ RSA key generation for digital signatures. The client generates a pair of RSA public and private keys and sends the public key to the server after completing the handshake. This key is then used to digitally sign messages, which helps prevent impersonation and protects against Man-In-The-Middle attacks. For regular message encryption and decryption, we utilize AES-GCM. This algorithm is based on the Advanced Encryption Standard (AES) and employs Galois/Counter Mode (GCM) to provide strong reliability and secure message decryption. Additionally, we implement HMAC for message authentication and verification. This ensures that messages have not been tampered with, effectively preventing message tampering attacks.

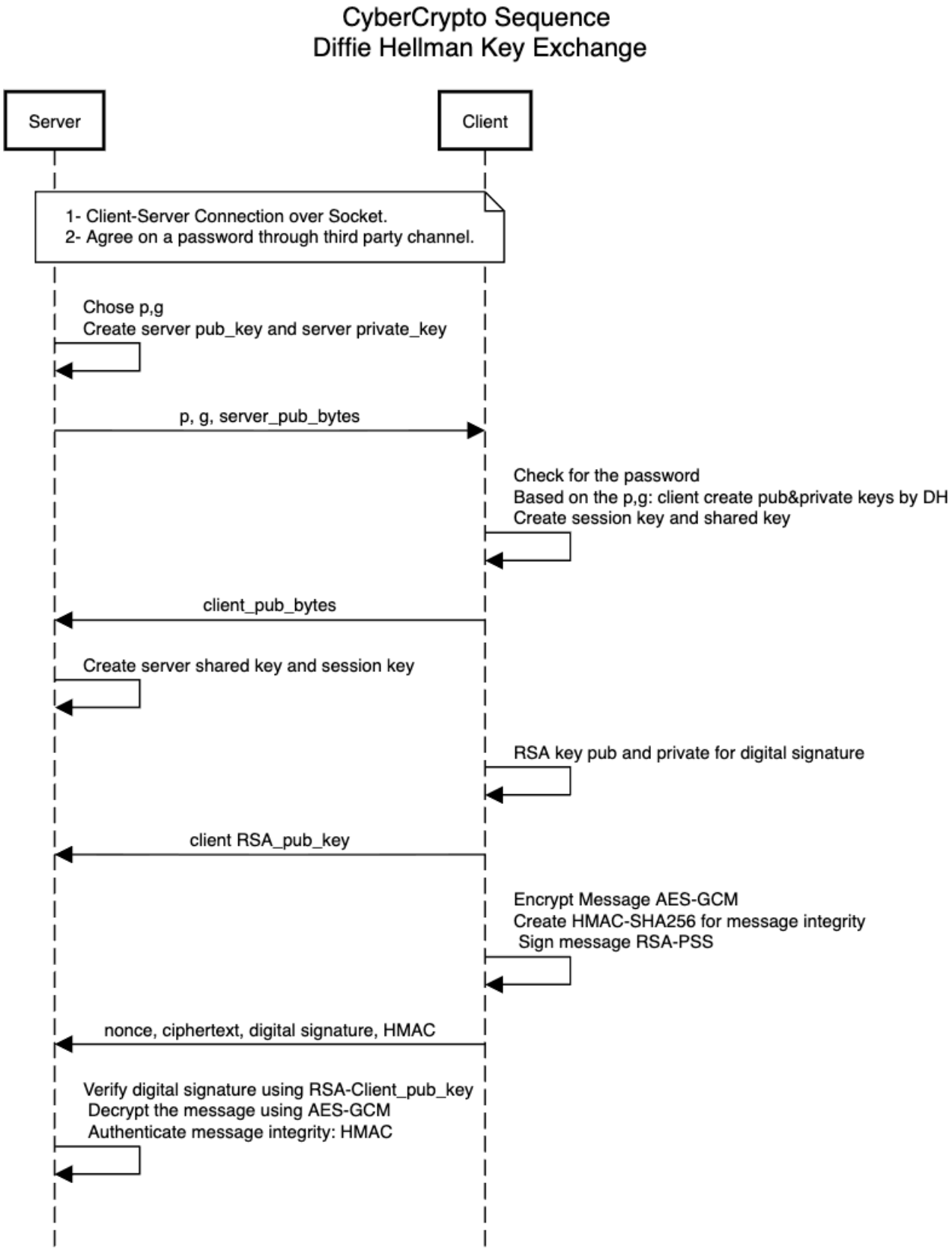


Figure 2: CyberCrypto Sequence Diagram

4.4 Key design decisions

For the encryption/decryption, we have employed AES-GCM with a key size of 256 bits. Galois/Counter Mode (GCM) is a high-performance symmetric-key block cipher that provides both confidentiality and data authenticity through authenticated encryption. However, if a nonce (a number used only once) is reused with the same key, an attacker can potentially recover the authentication key. This could allow them to forge messages and, in many cases, decrypt messages. This makes using GCM challenging in stateless protocols or when dealing with large volumes of data.

5 Security Analysis

5.1 Threat model

The threat model we have developed is based on four test cases:

1. No Diffie-Hellman key exchange (using random prime numbers) | Attack: Brute force.
2. HMAC verification | Attack: Message tampering.
3. Digital Signature (DS) verification | Attack: Man-in-the-Middle (MITM).
4. No Digital Signature (DS) and no HMAC.

5.2 Attack simulations

We have simulated four test cases based on the scenarios outlined above.

The first test case involved using a random prime number for communication instead of a Diffie-Hellman key exchange. To exploit this vulnerability, we employed a brute force attack mechanism to attempt to guess the 256-bit random key.

In the second test case, we simulated an attempt to tamper with a message by modifying the plaintext. We utilized the HMAC verification mechanism to detect whether the message had been tampered with.

The third simulation aimed to mimic a man-in-the-middle attack by attempting to alter the digital signature, thereby testing the concept of impersonation.

Finally, the last test case examined whether the use of digital signatures and HMAC algorithms affects the communication time. We measured the overhead introduced by these security mechanisms.

5.3 Security evaluation

We evaluated the security of CyberCrypto by checking whether the algorithms used can detect the threat model, which consists of three test cases and one overhead measurement.

Our findings indicate that all threats were successfully detected by the algorithms we employed (See appendicess figures for more information).

6 Performance Evaluation

The performance was evaluated based on the time required for CyberCrypto to complete the handshake, encrypt and decrypt messages, authenticate messages, and verify identities. Table 1 shows the results.

Table 1: Handshake and Attack Time Results | All time in second | Averaged on 4 attempts

No.	Test Case	Handshake Time	Attack Time	Verification Time
1	No DH	5.97	0.00896	0.00107
2	HMAC Verification	9.80	0.00744	0.000597
3	DS Verification	5.93	0.00789	0.000397
4	No DS, No HMAC	25.56	0.00356	0.000151

Table 2: CyberCrypto Scheme | All time in second | Averaged on 4 attempts

No.	Test Case	Handshake Time	Attack Time	Verification Time
1	CyberCrypto Scheme	6.56	0.00795	0.000660

7 Challenges and Issues

One of the most challenging issue we have faced is that the model was only tested on a small amount of input length, which does not cover all edge cases. Another issue is the variability of the handshake time; sometimes, the same run for the handshake takes longer than usual to establish a session key, but some times it takes less time.

7.1 How They Were Resolved

For the small number of input, we tried to maximize the message size limit from 10 MB to 100 MB. This allows the client to send much larger inputs without hitting the size cap. The limit remains as a security measure to prevent denial-of-service attacks.

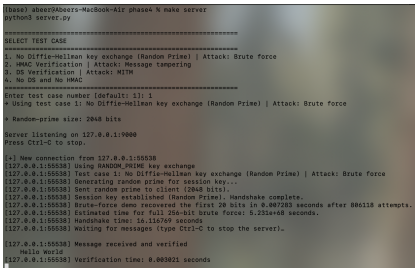
As for the time variation, we tried to test the simulated attacks in a stable environment, multiple times and take the average.

8 Conclusion

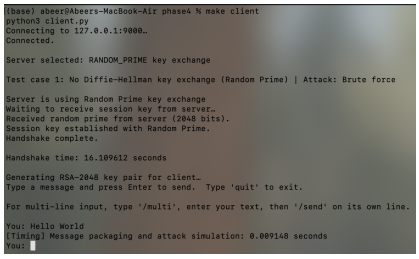
In this project, we designed and analyzed *CyberCrypto*, a secure communication system that combines key cryptographic concepts with practical implementation choices. The system integrates Diffie–Hellman key exchange for key session establishment, AES-GCM for confidential and authenticated encryption, RSA-based digital signatures for identity verification, and HMAC for message integrity. Together, these mechanisms provide a layered security design that addresses confidentiality, integrity, authentication, and resistance to common network-based attacks. Our security analysis showed that the proposed design can effectively detect and mitigate several realistic threats, including brute-force attempts, message tampering, and man-in-the-middle attacks. In addition, the performance evaluation demonstrated that these protections can be achieved with a better acceptable overhead, making the system suitable for secure end-to-end communication in practical settings.

Although the project achieved its main objectives, some limitations remain, particularly regarding the scale of testing and the variability of the handshake time. Future work can focus on expanding the evaluation to larger input, testing under more diverse attack scenarios, and exploring post-quantum cryptographic alternatives to strengthen long-term security. Overall, this project demonstrates how combining established cryptographic techniques with careful protocol design can produce a robust and efficient secure communication framework.

Appendices

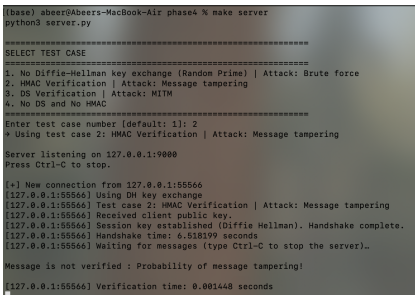


(a) Server

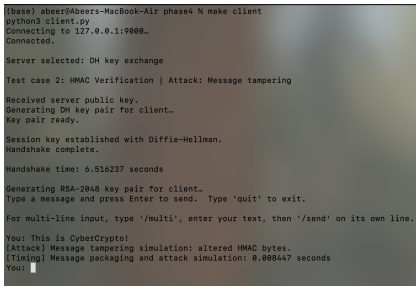


(b) Client

Figure 3: Test Case 1: No Diffie-Hellman key exchange (using random prime numbers

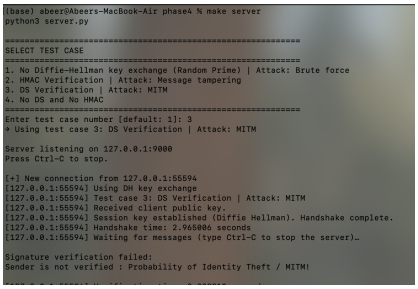


(a) Server

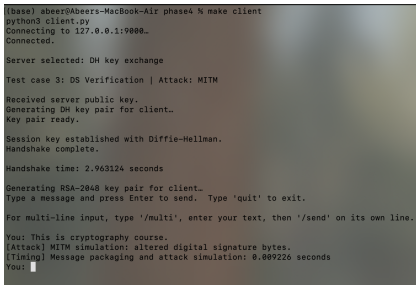


(b) Client

Figure 4: Test Case 2: HMAC verification

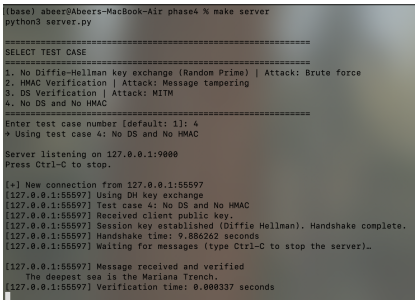


(a) Server

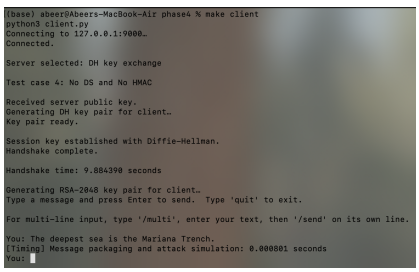


(b) Client

Figure 5: Test Case 3: Digital Signature (DS) verification



(a) Server



(b) Client

Figure 6: Test Case 4: No Digital Signature (DS) and no HMAC (Time Overhead Calculation)